

Project 2: Schema Linking for NL-to-SQL

CSE/DSC 234 – Data-Centric AI and AI Engineering, Spring 2026

Jay Chaudhary | Abhay Jain

June 4, 2026

1. Task and Dataset

Given a natural-language question q and the schema $\mathcal{S}$ of a relational database d , the goal is to predict a mapping $\hat{L} = \{t_i \mapsto [c_{i,1}, \dots]\}$ where each t_i is a referenced table and $c_{i,j}$ are the specific columns referenced by q . Tables that appear in a query with no specific column (e.g. `COUNT(*)`) are included with an empty list.

Evaluation. Score = $0.5 \times \text{Table Score} + 0.5 \times \text{Column Score}$, where each sub-score is the macro-average over questions of $(P + R + F_1)/3$.

Dataset. The SNAILS benchmark provides schemas for 17 real-world databases in Spider format [2]. The training split contains 301 question–schema–links triples; the validation split contains 101. Schemas are stored as JSON files with fields `table_names_original`, `column_names_original`, `primary_keys`, and `foreign_keys`.

2. Final Pipeline Architecture

The submitted pipeline consists of three scripts: `format_data.py` (dataset construction), `train.py` (LoRA fine-tuning), and `main.py` (inference). Table 1 summarises the final configuration that achieved a leaderboard score of **0.6117**.

Table 1: Final pipeline configuration.

Component	Setting
Base model	Qwen/Qwen2.5-1.5B-Instruct
LoRA targets	<code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>
LoRA rank / alpha	$r = 64$, $\alpha = 128$, dropout 0.1
Learning rate	5×10^{-5} , cosine schedule, 5% warmup
Batch / accumulation	1 per device $\times$ 4 grad. accum. steps (effective batch 4)
Epochs / max length	5 epochs, 2048 token sequence limit
Precision	bf16 mixed precision
Schema format	<code>TABLE(col*,col>RefTable)</code> — compact, PK=*, FK=>Ref
Prompt layout	Question first, then DB: and Schema: block
System prompt	35-token compact instruction (see Section 2)
Post-processing	<code>filter_to_schema</code> : removes hallucinated identifiers
Decoding	Greedy, repetition penalty 1.1, <code>max_new_tokens=512</code>

Data preparation (`format_data.py`). Each training example is a supervised chat record with three turns: *system* (task instruction), *user* (question + serialised schema), and *assistant* (gold schema-link dict as compact JSON). The schema is serialised as `TABLE(col*,col>RefTable,col)` — one line per table, no spaces after commas, with `*` marking primary keys and `>Table` marking foreign-key references. This encodes relational structure in minimal tokens while fitting all examples well within 2048 tokens.

Fine-tuning (`train.py`). LoRA adapters [1] are applied to all four attention projection matrices. Training uses `SFTTrainer` from TRL, orchestrated by RapidFire AI’s `Experiment + RFGGridSearch` API. The best checkpoint (lowest validation loss) is automatically copied to `./adapter/` after training.

Inference (main.py). The base model is loaded and the LoRA adapter is attached via `PeftModel.from_pretrained`. The prompt uses the same system and user templates as training. The raw generation is parsed with a tolerant extractor that falls back to regex matching on the first `{...}` block. `filter_to_schema` then removes any hallucinated table or column name not present in the actual schema, restoring exact casing via a case-insensitive lookup map.

Prompt design. The user message places the question *before* the schema so that attention can align question tokens with schema elements as they are processed:

Q: {question}

DB: {db_id}

Schema:

{schema_text}

The system prompt reads: *“Schema linker: given a question and DB schema, output JSON {table:[columns]}. Include tables used without specific columns as {table:[]}. Use exact schema casing. Output JSON only.”*

3. Experimentation Methodology

We adopted an iterative methodology: each iteration changed one knob (or a small related bundle), re-evaluated on the 101-question validation set, and was retained only if both table and column metrics improved. Table 2 lists all configurations considered.

Table 2: All configurations explored. Measured configs ran full train+eval; designed configs describe combinations reasoned about during iteration.

Config	r	LR	Epochs	Schema format	Status
C1: Baseline	16	2×10^{-4}	3	Simple TABLE(col1, col2)	measured
C2: very low LR	64	8×10^{-6}	5	Simple	measured
C3: JSON hierarchical	64	5×10^{-5}	5	Nested JSON with types/PK/FK	measured
C4: rich inline	64	5×10^{-5}	5	TABLE(col [PK, TEXT], ...)	measured
C5: r=64 simple	64	5×10^{-5}	5	Simple TABLE(col1, col2)	measured
C6: compact + pruning	64	5×10^{-5}	5	Compact PK/FK + keyword pruning	measured
C7: compact (final)	64	5×10^{-5}	5	Compact PK/FK, Q-first, no prune	measured (best)
C8: r=128 + col prompt	128	5×10^{-5}	5	Compact PK/FK, col-aware prompt	measured

Learning rate search (C1 $\rightarrow$ C5). With only 301 training examples and an effective batch of 4 (75 steps/epoch), the model is highly sensitive to the learning rate. At 2×10^{-4} and 1×10^{-4} , token accuracy oscillated between 0.42 and 0.56 across steps: the model peaked near step 50 (end of epoch 1) and then degraded, a classic sign of overshooting the loss surface on a small dataset. At 8×10^{-6} the model was too slow to converge (score 0.47). A rate of 5×10^{-5} with a cosine schedule and 5% warmup gave stable, monotonic validation-loss decrease across all 5 epochs.

Schema format search (C3, C4, C5, C6, C7). Schema serialisation turned out to be the single most impactful design axis. A nested JSON format (C3) frequently exceeded the 2048-token limit on databases with 20+ tables, causing truncation (score 0.20). A rich inline format annotating each column with its type and key role (C4) conveyed more information but added enough tokens to confuse the model (score 0.45). The simple TABLE(col1, col2) format (C5) performed best among explicit formats (0.54), which motivated the compact PK/FK variant: encoding `col*` for primary keys and `col>Ref` for foreign keys conveys relational join structure in almost no extra tokens.

Question-first prompt and schema pruning (C6, C7). Moving the question before the schema (C7) allows the decoder’s attention to attend to question tokens while processing each schema line. We also experimented with inference-time schema pruning (C6): retaining only tables whose names or column names share keywords with the question, then expanding via foreign-key neighbours. This reduced median context length but caused the table score to drop from 0.683 to 0.552, because pruning at inference – without knowledge of the gold tables – occasionally hid tables the model needed to predict. Pruning was removed; all tables are shown at both training and inference time.

4. Results and Analysis

Table 3 shows all measured scores on the 101-question validation set. Configs C3, C4, and C6 include sub-scores where available.

Table 3: Measured scores on the 101-question validation set. Dashes indicate runs where sub-scores were not recorded.

Config	Key change	r	LR	Ep.	Table	Column	Score
C1 Baseline	$r = 16$, simple, 2×10^{-4}	16	2×10^{-4}	3	–	–	0.52
C2 low LR	$lr = 8 \times 10^{-6}$	64	8×10^{-6}	5	–	–	0.47
C3 JSON hier.	Hierarchical JSON schema	64	5×10^{-5}	5	–	–	0.20
C4 rich inline	Type + PK/FK verbose	64	5×10^{-5}	5	–	–	0.45
C5 r64 simple	$r = 64$, simple format	64	5×10^{-5}	5	–	–	0.54
C6 w/ pruning	Compact + Q-first + pruning	64	5×10^{-5}	5	0.552	0.484	0.518
C7 final	Compact + Q-first, no prune	64	5×10^{-5}	5	0.683	0.541	0.612
C8 r=128 col-prompt	$r=128$, col-aware sys. prompt	128	5×10^{-5}	5	0.685	0.529	0.607

Detailed metrics for the best run (C7). At the table level: $P_T = 0.699$, $R_T = 0.682$, $F1_T = 0.667$. At the column level: $P_C = 0.577$, $R_C = 0.532$, $F1_C = 0.514$. Zero schema-invalid identifiers were predicted, confirming that `filter_to_schema` eliminates all hallucinations.

Table vs. column score gap. Across all runs, the table score consistently exceeds the column score by roughly 0.14 points. This reflects an inherent asymmetry: the model must identify a much smaller set of tables (typically 1–3 per question) compared to columns (often 4–8). Column prediction additionally requires understanding *implicit* references – join keys and filter predicates that are not lexically present in the question text.

Why compact format helps. The compact `TABLE(col*,col>Ref)` encoding achieves two goals simultaneously. First, it reduces token count per table line by roughly 15–20% compared to spaced, verbose formats, leaving more context budget to represent all tables without truncation. Second, the `*` and `>Ref` markers encode relational structure (primary and foreign keys) that helps the model identify join paths between tables, improving identification of join-key columns – a known hard case for column recall.

Why pruning hurt table score. Pruning reduced median context length but introduced a train/inference mismatch: during training, gold tables are always included, but at inference the keyword-based pruner has no oracle knowledge. A table whose name does not share lexical tokens with the question (e.g. a lookup table referenced only through a foreign key chain) was silently hidden, making it impossible for the model to predict. The -0.13 table score swing (C6 vs. C7) directly quantifies this effect.

Result of C8. Doubling the LoRA rank to $r = 128$ ($\alpha = 256$) and adding an explicit column-type enumeration to the system prompt yielded a score of 0.6074 (Table: 0.685, Column: 0.529) — marginally

below C7’s 0.6117. The table score held steady while column recall fell slightly, suggesting the column-aware prompt alone does not overcome the implicit-reference gap at this data scale. C7 remains the best configuration.

5. AI Tools Statement

Claude (Anthropic) was used as an assistive tool during the development of this project, primarily for code generation, debugging, and iterating on the schema serialisation and prompt formats. All experimental design decisions, hyperparameter choices, result interpretation, and final written content are the responsibility of the author.

References

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- [2] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP 2018*.